BIBLIOTECA FANTÁSTICA

1. CONTEXTO

Una biblioteca especializada en literatura fantástica necesita una aplicación web para gestionar su catálogo de libros y sus autores.

La aplicación tendrá una interfaz web desde la que se podrán consultar los libros y eliminar aquellos que ya no formen parte del catálogo. También permitirá consultar los autores, registrar nuevos autores y modificar sus datos.

La aplicación se desarrollará utilizando PHP y MySQL, siguiendo el patrón MVC y separando el acceso a datos de la lógica de negocio y de la presentación.

2. OBJETIVOS

- Desarrollar una API REST con PHP.
- Conectar PHP con una base de datos MySQL.
- Procesar peticiones HTTP y devolver respuestas en formato JSON.
- Implementar las operaciones CRUD.
- Validar los datos recibidos antes de almacenarlos.
- Gestionar errores y devolver códigos de estado HTTP adecuados.
- Comprobar el funcionamiento de la API utilizando el cliente proporcionado.

2. OPERACIONES QUE DEBE IMPLEMENTAR LA APLICACIÓN

Tabla libros

Consultar y eliminar

- Mostrar todos los libros registrados.
- Eliminar un libro seleccionado.
- Solicitar confirmación antes de eliminarlo.
- Actualizar la lista después de eliminarlo.

Tabla autores

Consultar, crear y actualizar

- Mostrar todos los autores registrados.
- Crear un autor mediante un formulario.
- Seleccionar un autor y cargar sus datos en el formulario.
- Modificar sus datos y guardar los cambios.
- Actualizar la lista después de cada operación.

Restricción: no se debe implementar la operación de crear ni actualizar libros, ni la operación de eliminar autores. Cada tabla tendrá exclusivamente las operaciones asignadas.

3. BASE DE DATOS

La base de datos se llamará **biblioteca_fantastica** y tendrá dos tablas relacionadas:

- **autores:** almacena la información de los autores.
- **libros:** almacena los libros y el autor al que pertenece cada uno.

Ejecuta el siguiente script en phpMyAdmin.

```
CREATE DATABASE biblioteca_fantastica
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE biblioteca_fantastica;

CREATE TABLE autores (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    nacionalidad VARCHAR(60) NOT NULL,
    fecha_nacimiento DATE NOT NULL
);
```

```
CREATE TABLE libros (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    genero VARCHAR(50) NOT NULL,
    anio_publicacion INT NOT NULL,
    id_autor INT NOT NULL,
    FOREIGN KEY (id_autor)
        REFERENCES autores(id)
);
USE biblioteca_fantastica;

INSERT INTO autores
    (nombre, nacionalidad, fecha_nacimiento)
VALUES
    ('J.R.R. Tolkien', 'Britanica', '1892-01-03'),
    ('Ursula K. Le Guin', 'Estadounidense', '1929-10-21'),
    ('Laura Gallego', 'Española', '1977-10-11'),
    ('George R.R. Martin', 'Estadounidense', '1948-09-20'),
    ('Terry Pratchett', 'Britanica', '1948-04-28');

INSERT INTO libros
    (titulo, genero, anio_publicacion, id_autor)
VALUES
    ('El Hobbit', 'Fantasia', 1937, 1),
    ('El Senor de los Anillos', 'Fantasia', 1954, 1),
    ('Un mago de Terramar', 'Fantasia', 1968, 2),
    ('Memorias de Idhun', 'Fantasia', 2004, 3),
    ('Juego de Tronos', 'Fantasia epica', 1996, 4),
    ('Mort', 'Fantasia comica', 1987, 5),
    ('La historia interminable', 'Fantasia', 1979, 3);
```

4. FUNCIONALIDADES

La API deberá implementar las siguientes operaciones para gestionar los libros y los autores de la biblioteca.

Tabla libros: consultar y eliminar

- **Ver todos:** devolver un listado de todos los libros registrados, incluyendo su título, género, año de publicación y nombre del autor.
- **Eliminar:** eliminar un libro de la base de datos a partir de su identificador.
- **Actualizar listado:** después de eliminar un libro, la interfaz deberá mostrar la lista actualizada.

Tabla autores: consultar, crear y actualizar

- **Ver todos:** devolver un listado de todos los autores registrados, incluyendo su nombre, nacionalidad y fecha de nacimiento.
- **Insertar:** registrar un nuevo autor con los datos recibidos.
- **Modificar:** actualizar los datos de un autor existente a partir de su identificador.
- **Actualizar listado:** después de crear o modificar un autor, la interfaz deberá mostrar los datos actualizados.

5. CONTRATO DE LA API

La API se dividirá en dos puntos de entrada:

- **/api/libros.php**
- **/api/autores.php**

API de libros: /api/libros.php

GET

Parametro: Ninguno.

Funcionalidad: Obtener todos los libros, incluyendo el nombre de su autor.

DELETE

Parametro: ?id=1

Funcionalidad: Eliminar el libro con identificador 1.

API de autores: /api/autores.php

GET

Parametro: Ninguno.

Funcionalidad: Obtener todos los autores.

POST

Parametro: JSON en el cuerpo.

Funcionalidad: Crear un nuevo autor.

PUT

Parametro: ?id=1 y JSON en el cuerpo

Funcionalidad: Modificar el autor con identificador 1.

Todas las respuestas deberán estar en formato JSON y utilizar códigos de estado HTTP apropiados, como 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 405 Method Not Allowed y 500 Internal Server Error

La API deberá rechazar los métodos HTTP que no estén permitidos para cada recurso.

6. VALIDACIONES

Validaciones de autores

- El nombre no esté vacío y no supere los 100 caracteres.
- La nacionalidad no esté vacía y no supere los 60 caracteres.
- La fecha de nacimiento sea válida y tenga el formato YYYY-MM-DD.
- Los datos recibidos sean válidos antes de insertar o actualizar un autor.
- El identificador corresponda a un autor existente cuando se vaya a modificar.
- No se pueda modificar un autor que no exista en la base de datos.

Validaciones de libros

- El identificador recibido para eliminar un libro sea válido y corresponda a un libro existente.
- Si se intenta eliminar un libro que no existe, la API deberá devolver un error adecuado.
- La base de datos deberá mantener la integridad de la relación entre libros y autores mediante la clave foránea definida en el script SQL.

Validaciones generales

- Utilizar consultas preparadas para evitar inyecciones SQL.
- Devolver respuestas JSON tanto en las operaciones correctas como en los errores.
- Utilizar códigos HTTP adecuados para identificar los errores.
- Gestionar los errores de conexión y de ejecución de consultas sin mostrar información interna de la base de datos al usuario.
- No permitir operaciones no contempladas en el contrato de cada API.

Importante: no es necesario desarrollar el HTML, el CSS ni el JavaScript de la interfaz, ya que se proporcionan preparados. Debes implementar únicamente la parte del servidor en PHP, incluyendo los modelos, los DAO, los controladores y los puntos de entrada de ambas API, respetando la arquitectura MVC y el contrato definido en este enunciado.

